

An Improved On-line Node Ranking Algorithm of Trees

Yung-Ling Lai and Yi-Ming Chen

Department of Computer Science and Information Engineering

National Chiayi University, Taiwan

{yllai, s0914752}@mail.ncyu.edu.tw

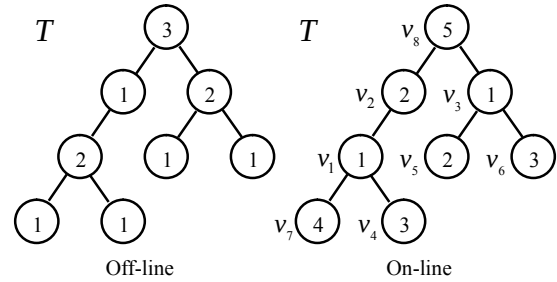
Abstract

A k -ranking of a graph $G = (V, E)$ is a mapping $C: V \rightarrow \{1, 2, \dots, k\}$ such that for every path between any two vertices u and v , with $C(u) = C(v)$, there is a vertex w on the path with $C(w) > C(u) = C(v)$. The value $C(v)$ is called the rank or color of the vertex v . A node ranking problem (also called ordered coloring problem [7]) is the problem of finding minimum k such that G has k -ranking. In the online version of this problem, the vertices $v_1, v_2, \dots, v_n$ are given one by one in an arbitrary order and only the edges of the induced subgraph $\langle \{v_1, v_2, \dots, v_i\} \rangle_G$ are known when the rank of v_i has to be chosen. [9] provided a node ranking algorithm of trees for online version with time $O(n^3)$ and space $O(n^2)$. This paper presents an $O(n^2)$ time complexity and $O(n)$ space complexity online node ranking algorithm for arbitrary trees.

1 Introduction

A graph $G = (V, E)$ is called k -rankable if there is a mapping $C: V \rightarrow \{1, 2, \dots, k\}$ such that for any path of G with end-vertices u, v such that $C(u) = C(v)$, there is a vertex w lies on the path with $C(w) > C(u) = C(v)$. The value $C(v)$ is called the rank or color of the vertex v . The ranking number $\chi_r(G)$ of graph G is the smallest integer k such that G is k -rankable. The node ranking problem (also called ordered coloring problem [7]) is the problem of finding the ranking number of graph G . For an arbitrary graph G , the decision version of the node ranking problem is NP-complete [1], in fact, this problem remains NP-Complete even with restriction of the graph to be cobipartite graphs [13]. The node ranking is an interested problem and it can be applied to communication network design [3, 6, 12, 15], computing Cholesky factorizations of matrices in parallel [1, 5, 11], finding the minimum-height elimination tree of a graph [3, 4], and VSLI Layout [10, 16] etc. If the vertex of the graph is given one by one in an arbitrary order, and

when assigning the rank of vertex v_i , only the edges of the induced subgraph $\langle \{v_1, v_2, \dots, v_i\} \rangle_G$ are known, this is considered as the online version of the node ranking problem. In the online version, we want to assign the rank of the vertex in real time and the rank cannot be changed later on. We use $\chi_r^*(G)$ to denote the online ranking number of graph G . Figure 1 shows an example of node ranking of a tree with both off-line and on-line versions.

Figure 1: Off-line and on-line ranking of tree T .

Node ranking problem has been studied since 1980s. Most of them studied off line version where the whole graph is given beforehand. [7] showed that $\chi_r(P_n) = \lfloor \log n \rfloor + 1$ for $n \geq 1$ and [2] proved $\chi_r(C_n) = \lfloor \log(n-1) \rfloor + 2$ for $n \geq 3$. An upper bound of the ranking number of an arbitrary tree with n nodes was given by [6], they proposed an $O(n \log n)$ time optimal node ranking algorithm of trees, which was further improved to $O(n)$ by [14]. For the online version, [2] gave bounds of the node ranking number of paths and cycles as $\lfloor \log n \rfloor + 1 < \chi_r^*(P_n) < 2\lfloor \log n \rfloor + 1$ and $\lfloor \log(n-1) \rfloor + 2 < \chi_r^*(C_n) < 2\lfloor \log(n-1) \rfloor + 1$ respectively. [8] gave an online ranking algorithm for trees using CREW PRAM model with $O(n^3 / \log^2 n)$ processors. And [9] provided an optimal online ranking algorithm for stars and an $O(n^3)$ time algorithm for arbitrary trees. This paper described an $O(n^2)$ time online ranking algorithm for arbitrary trees.

2 An on-line node ranking algorithm for trees

In this section, we propose an online ranking algorithm for arbitrary trees. We use an array `available[]` to indicate the available rank assignment for the new vertex v_j . `available[i]=0` when rank i is available for v_j and `available[i]>0` otherwise. When a new vertex v is given, procedure `Modify` will be called recursively using `MaxValue` to update the array `available[i]`. After reviewing all existing vertices, assign the first possible i (i.e. `available[i]=0`) which appears after all possible j such that `available[j]>1` to be the rank of v .

Algorithm Online_Ranking_Trees

Input:

n : total number of vertices of the input tree T .

$v_1, v_2, \dots, v_n$: input one after another by point out the edges of the induced subgraph $\langle \{v_1, v_2, \dots, v_i\} \rangle_T$ incident with v_i .

Output: An online rank assignment of T .

Method:

```

For i = 1 to n do {
  Read the edges incident with  $v_i$  and  $v_j$  for
     $1 \leq j < i$ 
  For j = 1 to n do // initial array available[]
    available[j]=0;

  // call procedure Modify to modify available[]
  Modify( $v_i$ ,  $v_i$ , 0);

  // find the maximum index r such that
  // available[r]>1
  r=1;
  For j = 1 to n do
    If (available[j]>1) Then r = j;

  // find the first available rank and assign it to  $v_i$ 
  For j = r to n do
    If (available[j] = 0) then {
       $C(v_i) = j$ ;
      Break;}
    }

```

End of Algorithm Online_Ranking_Trees

Procedure Modify(v , w , $MaxValue$)

```

for every vertex  $u$  adjacent to  $v$  except  $w$  do {
  if  $C(u) > MaxValue$  then {
    available [ $C(u)$ ] = available [ $C(u)$ ] +1;
  //  $C(u)$  is the max rank on the path from  $v$  to  $u$ 
  Modify( $u$ ,  $v$ ,  $C(u)$ );
  }
  //  $MaxValue$  is the max rank from  $v$  to  $u$ 
  else Modify( $u$ ,  $v$ ,  $MaxValue$ );}

```

End of Procedure Modify

Theorem 1: Algorithm `Online_Rnaking_Trees` has $O(n^2)$ time complexity and $O(n)$ space complexity.

First we analyze the time complexity of the algorithm. Since a tree with n vertices has exactly $n-1$ edges, each time when insert a new vertex v_i , the Procedure `Modify` will be called no more than $i-1$ times, so after n vertices inserted, the Procedure `Modify` will be called at most $1+2+\dots+(n-1) = O(n^2)$ times. All other for-loops in the algorithm have $O(n)$ computation time for each v_i 's insertion, and there are three such loops, so the total time complexity (with procedure `modify`) will be $O(n^2) + O(n^2) + O(n^2) + O(n^2) = O(n^2)$. For the space complexity, we use adjacency list to store the edges plus one size n array (`available[]`), the total space complexity then is $O(n) + O(n) = O(n)$.

Theorem 2: Algorithm `Online_Rnaking_Trees` produces a valid online ranking of the input tree.

Proof: There are three cases when a new vertex v_i is inserted:

Case 1: v_i is adjacent to no vertex v_j for $1 \leq j < i$. In this case, no value in `available[]` will be changed so $C(v_i)=1$. Since it is an isolated vertex, it satisfies the ranking rule.

Case 2: v_i is adjacent to only one vertex v_j for $1 \leq j < i$. Let T^* be the connected subgraph of $\langle \{v_1, v_2, \dots, v_i\} \rangle_T$ containing v_i . We show this case by mathematical induction on $|V(T^*)|$.

Basis: When $|V(T^*)|=2$, v_j for $1 \leq j < i$ is an isolated vertex in $\langle \{v_1, v_2, \dots, v_{i-1}\} \rangle_T$ and is adjacent to v_i . Procedure `Modify` will increase the `available[ $C(v_j)$ ]` by 1 hence $C(v_i)$ will not get the same value as $C(v_j)$.

Inductive Hypothesis: Assume that the rank assignments for the induced subgraph $\langle \{v_1, v_2, \dots, v_i\} \rangle_T$ satisfies the ranking rule for all subgraph T^* with $|V(T^*)| < k$.

Inductive Step: Let $|V(T^*)|=k$ consider $T' = T^* - v_i$, then $|V(T')| < k$ by inductive hypothesis, T' satisfy the ranking rule. Let v_i connect to T' by v_j as shown in figure 2. Now we have to check for all vertices $u \in T'$ with $C(u) = C(v_i)$ there exists a vertex w lies on $u - v_i$

path such that $C(w) > C(v_i)$.

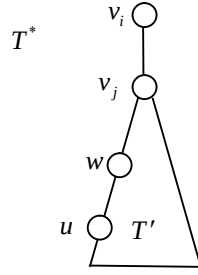


Figure 2: v_i and T' is connected by v_j .

Consider a path $v_i - u$ for $u \in T'$ with $C(v_i) = C(u)$, assume during visit the vertices on the path in procedure Modify, the array `available[]` has been changed when it reached vertices $u_1, u_2, u_3 \dots u_{n'}$ in order, since `available[C(x)]` will be increased only when $C(x) > \text{MaxValue}$, we must have $C(u_j) < C(u_{j+1})$ for $1 \leq j \leq n'$ and all the vertices on the $u_j - u_{j+1}$ path have ranks lower than $C(u_j)$ for $1 \leq j \leq n'$ as shown in figure 3. Since for each vertex u_j , `available[C(u_j)]` has been increased we know that $C(v_i) \neq C(u_j)$ for $1 \leq j \leq n'$. In case of $C(v_i) < C(u_1)$, since u_1 is the only adjacent vertex of v_i , u_1 must lie on every $v_i - u$ path. If $C(u_j) < C(v_i) < C(u_{j+1})$ for some j , $1 < j \leq n' - 1$, then u_j must lie on $v_i - u$ path. If $C(u_{n'}) < C(v_i)$, then $C(u) > \text{MaxValue}$ and `available[C(u)]` will be increased hence $C(v_i) \neq C(u)$. Then we have case 2 satisfies the ranking rule.

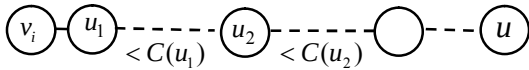


Figure 3: $C(v_i) = C(u)$

Case 3: v_i is adjacent to two or more vertices in $\langle \{v_1, v_2, \dots, v_i\} \rangle_T$. We have already showed that all paths in $\langle \{v_1, v_2, \dots, v_i\} \rangle_T$ with v_i as one end-vertex satisfy the ranking rule in case 2, so we just need to show that all paths with v_i as an internal node satisfy the ranking rule. For each of such path can be viewed as two paths which are connected by v_i (see figure 4 as an example). Consider path $u_k - w_l$ with $C(u_k) = C(w_l)$, like case 2, assume v_i is adjacent to vertices u_1 and w_1 , and $P_u : v_i, u_1, u_2 \dots u_k$, $P_w : v_i, w_1, w_2, \dots w_l$ indicate the

paths start from v_i to u_k and v_i to w_l respectively as shown in figure 4..

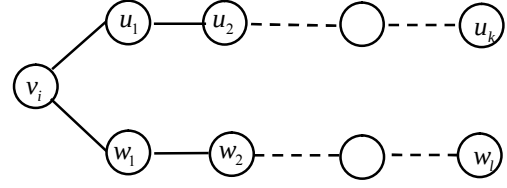


Figure 4: v_i is an internal node of a path $u_k - w_l$

Assume during visit the vertices on the paths P_u and P_w in procedure Modify, the array `available[]` have been changed when it reached vertices $u_1 = u'_1, u'_2, u'_3 \dots u'_k$ and $w_1 = w'_1, w'_2, w'_3 \dots w'_l$ in order respectively. Since `available[C(x)]` will be increased only when $C(x) > \text{MaxValue}$, we must have $C(u'_j) < C(u'_{j+1})$ and $C(w'_t) < C(w'_{t+1})$ for $1 \leq j < k'$ and $1 \leq t < l'$, and all the vertices on the $u'_j - u'_{j+1}$ path have lower rank than $C(u'_j)$ for $1 \leq j < n'$, all the vertices on the $w'_t - w'_{t+1}$ path have rank lower than $C(w'_t)$ for $1 \leq t < l'$ as illustrated in figure 5. If either $u_k \neq u'_k$ or $w_l \neq w'_l$ (without loss of generality, assume $u_k \neq u'_k$), that is $C(u_k) < C(u'_k)$, then u'_k is a vertex on $u_k - w_l$ path which has higher rank than u_k and hence satisfy the ranking rule. If $u_k = u'_k$ and $w_l = w'_l$, then since $C(u_k)$ is the maximum rank in P_u and $C(w_l)$ is the maximum rank in P_w , both of them will update `available[C(u_k)]` which makes `available[C(u_k)]` ≥ 2 and results $C(v_i) > C(u_k)$, that is v_i is a vertex on the path with higher rank than u_k and so satisfy the ranking rule.

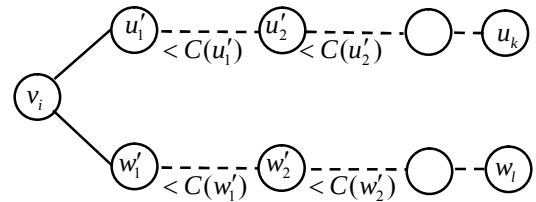


Figure 5: The ranking relation between path $u_k - w_l$

In all cases we have shown that Algorithm `Online_Rnaking_Trees` produces a valid online ranking of the input tree. $\square$

3 Conclusion

There are several papers discussing the node ranking. In this paper, we presented an online ranking algorithm for arbitrary trees. Both time and space complexities of our algorithm are lower than [9] while we get the same ranking result as [9].

References

- [1] H. L. Bodlaender, J. R. Gilbert, H. Hafsteninsson and T. Kloks, "Approximating treewidth, pathwidth, frontsize and shortest elimination tree," *Journal of Algorithms* 18, pp. 238-255, 1995.
- [2] E. Bruoth and M. Horňák, "On-line ranking number for cycles and paths," *Discussiones Mathematicae, Graph Theory* 19, pp. 175-197, 1999.
- [3] P. de la Torre, R. Greenlaw, A. A. Schäffer, "Optimal ranking of trees in polynomial time," *Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms*, pp. 138-144, 1993.
- [4] J. S. Deogun, T. Kloks, D. Kratsch, and H. Muller, "On vertex ranking for permutation and other graphs," *Proc. of the 11th Annual Symposium on Theoretical Aspects of Computer Science*, P. Enjalbert, E.W. Mayr, K.W. Wagner, *Lecture Notes in Computer Science* 775, Springer-Verlag, Berlin, pp. 747-758, 1994.
- [5] I. S. Duff and J. K. Reid, "The multifrontal solution of indefinite sparse symmetric linear equations," *ACM Transactions on Mathematical Software* 9 (1983), pp. 302-325.
- [6] A.V. Iyer, H.D. Ratliff and G. Vijayan, "Optimal node ranking of trees," *Information Processing Letters* 28, pp. 225-229, 1988.
- [7] M. Katchalski, W. McCuaig, and S. Seager, "Ordered colorings," *Discrete Mathematics* 142, pp. 141-154, 1995.
- [8] Chia-wei Lee and Justie Su-tzu Juan, May 2005, "Parallel Algorithm for On-Line Ranking in Trees," *Proc. of the 22nd Workshop on Combinatorial Mathematics and Computational Theory*, National Cheng Kung University, Tainan, Taiwan, May 20-21, pp. 151-156, 2005.
- [9] Chia-wei Lee and Justie Su-tzu Juan, Jun. 2005, "On-Line Ranking Algorithms for Trees," *Proc. of Int. Conf. on Foundations of Computer Science*, Monte Carlo Resort, Las Vegas, Nevada, USA, June 27-30, pp. 46-51, 2005.
- [10] C.E. Leiserson, "Area-efficient graph layouts (for VLSI)," *Proceedings of the 21st Annual IEEE Symposium on Foundations of Computer Science*, pp. 270-281, 1980.
- [11] J.W.H. Liu, "The role of elimination trees in sparse factorization," *SIAM Journal of Matrix Analysis and Applications* 11, pp. 134-172, 1990.
- [12] J. Nevins, D. Whitney (Eds.), "Concurrent Design of Products and processes," McGraw-Hill, New York, 1989.
- [13] A. Pothen, "The complexity of optimal elimination trees," *Technical Report CS-88-13*, Pennsylvania State University, University Park, PA, 1988.
- [14] A. A. Schäffer, "Optimal node ranking of trees in linear time," *Information Processing Letters* 33, pp. 91-96, 1989.
- [15] P. Scheffler, "Node ranking and searching on cographs (Abstract)," in: U. Faigle and C. Hoede (Eds.), *3rd Twente Workshop on Graph and Combinatorial Optimization*, 1993.
- [16] A. Sen, H. Deng, and S. Guha, "On a graph partition problem with application to VLSI layout," *Information Processing Letters* 43, pp. 87-94, 1992.